

1. DE L'ALÉATOIRE

La bibliothèque C permet de générer des entiers aléatoires pris dans l'ensemble $\llbracket 0, \text{RAND_MAX} \rrbracket$. On suppose que ce tirage est uniforme.

Le générateur aléatoire doit être initialisé avec une graine avant d'être utilisé pour la première fois. La séquence de nombres aléatoires est fixée par la graine, si on initialise à nouveau le générateur avec la même graine, on obtiendra à nouveau la même séquence de nombres.

```
#include <stdlib.h>

/* Obtention d'un entier aléatoire */
int rand(void);
/* Initialisation du générateur */
void srand(unsigned int seed);
```

C

Une pratique fréquente consiste à utiliser l'heure actuelle pour initialiser le générateur :

```
#include <time.h>

void init_alea() {
    srand(time(NULL));
}
```

C

Ce générateur convient très bien pour des algorithmes probabilistes, mais il n'est pas du tout assez robuste par exemple pour réaliser de la cryptographie ou générer des données nécessitant de vraies propriétés d'aléatoire.

Question 1. On veut simuler une loi uniforme sur un ensemble $\llbracket 0, N \rrbracket$. Pourquoi le code suivant est-il incorrect ?

C

```
int uniforme(int n) {
    return rand() % n;
}
```

2. GÉNÉRER UN GRAPHE ALÉATOIRE, MODÈLE D'ERDŐS ET RÉNYI

Soit p un réel strictement entre 0 et 1. On représente les graphes par des listes d'adjacence. On note $G_{n,p}$ un graphe à n sommets tel que chaque arête est présente avec probabilité p .

Question 2. Écrire une fonction `graphe_genere` qui génère un tel graphe.

Question 3. Donner l'espérance du nombre d'arêtes de $G_{n,p}$.

On aura besoin de considérer que les arêtes d'un sommet sont réparties uniformément au hasard dans chaque liste d'adjacence. Pour cela, on va mélanger les listes d'adjacence.

Question 4. Écrire une fonction `melange_liste` qui mélange en place une liste d'adjacence. (On pourra utiliser par exemple l'algorithme de Knuth, Fisher et Yates.)

Question 5. Modifier la fonction `graphe_genere` afin qu'elle renvoie des listes d'adjacence mélangées.

Erdős et Rényi proposent un deuxième modèle de graphes aléatoires, noté $G_{n,m}$ où n est un nombre de sommets fixé et m est le nombre d'arêtes à créer. Un tel graphe est obtenu en tirant exactement m arêtes parmi les $\binom{n}{2}$ arêtes possibles. Ceci peut se faire en

tirant les arêtes les unes après les autres.

Question 6. On considère $\mathcal{E} = \{(a, b) \in \llbracket 0, n-1 \rrbracket^2 \mid a > b\}$ et φ la fonction définie sur $\mathcal{E}$ par $\varphi(a, b) = \binom{b}{1} + \binom{a}{2}$. Montrer qu'elle réalise une bijection (quel est l'ensemble image ?).

Question 7. Écrire une fonction `graphe_genere_aretes` qui génère un tel graphe. On prendra soin de bien choisir uniformément les arêtes.

Dans la suite, on travaillera avec un graphe $G_{n,p}$.

3. UN PREMIER ALGORITHME DE CHEMIN HAMILTONIEN

On considère un chemin élémentaire du graphe, formé des sommets $v_1, \dots, v_k$ dans cet ordre. Sur un tel chemin, on va réaliser deux opérations :

- l'ajout d'un sommet en fin de chemin,
- la *rotation* `Rotation( $v_i, v_k$ )`. C'est l'opération qui transforme ce chemin en $v_1, \dots, v_i, v_k, v_{k-1}, \dots, v_{i+1}$, c'est-à-dire qu'elle retire l'arête entre v_i et v_{i+1} et ajoute au chemin l'arête entre v_i et v_k .

Question 8. Proposer une structure pour représenter un chemin qui permet de réaliser ces opérations. Implémenter la fonction `rotation` correspondante. (Un $O(n)$ est facile à voir. Je ne sais pas si on peut faire moins.)

Question 9. Implémenter la fonction qui permet d'ajouter un sommet à la fin du chemin.

À chaque étape de l'algorithme, on garde en mémoire pour chaque sommet une liste d'arêtes adjacentes à ce sommet et qui n'ont pas encore été utilisées. On note, pour chaque sommet v , `unused( $v$ )` cette liste. Initialement, elle correspond à la liste d'adjacence de v .

```
for all  $v \in V$  do
  unused( $v$ )  $\leftarrow \{(v, u) \mid (v, u) \in E\}$ 
end for
Prendre un sommet au hasard, en faire la tête du chemin
while on n'a pas trouvé de solution do
  Soit  $(v_1, \dots, v_k)$  le chemin actuel
  if unused( $v_k$ ) =  $\emptyset$  then
    Échouer
  end if
  Soit  $(v_k, u)$  la première arête de unused( $v_k$ )
  Retirer  $(v_k, u)$  de unused( $v_k$ )
  Retirer  $(u, v_k)$  de unused( $u$ )
  if  $u \notin \{v_1, \dots, v_{k-1}\}$  then
    Ajouter  $u$  à la fin du chemin
  else
    Soit  $i$  tel que  $u = v_i$ 
    Faire une rotation entre  $v_k$  et  $v_i$ . Le dernier sommet du chemin devient  $v_{i+1}$ .
  end if
end while
```

Question 10. Implémenter cet algorithme. Comment sait-on si on a terminé ?

4. INTERMÈDE MATHÉMATIQUE

Si X est une variable aléatoire, si $a \geq 0$ et si $t > 0$, alors l'inégalité de Chernoff dit :

$$P(X \geq a) \leq \frac{E(e^{tX})}{e^{ta}}$$

$$P(X \leq a) \leq \frac{E(e^{-tX})}{e^{-ta}}$$

Cela sert souvent en informatique, ainsi que les inégalités de Markov et de Bienaymé-Tchebychev.

5. PREUVE DE VALIDITÉ

L'algorithme précédent n'est pas facile à analyser car les choix effectués à une étape conditionnent les choix suivants. On introduit un algorithme modifié, qui effectue des étapes qui ne servent pas à améliorer le chemin trouvé, mais qui garantissent que les étapes de l'algorithme deviennent indépendantes.

```

Prendre un sommet au hasard, en faire la tête du chemin
while on n'a pas trouvé de solution do
  Soit  $(v_1, \dots, v_k)$  le chemin actuel
  if avec probabilité  $1/n$  then
    Retourner le chemin
  else if avec probabilité  $\text{card}(\text{used}(v_k))/n$  then
    Choisir une arête  $(v_k, v_i)$  au hasard dans  $\text{used}(v_k)$ 
    Faire la rotation  $(v_k, v_i)$ , la fin du chemin devient  $v_{i+1}$ 
  else
    Soit  $(v_k, u)$  la première arête de  $\text{unused}(v_k)$ 
    Retirer  $(v_k, u)$  de  $\text{unused}(v_k)$ 
    Retirer  $(u, v_k)$  de  $\text{unused}(u)$ 
    if  $u \notin \{v_1, \dots, v_{k-1}\}$  then
      Ajouter  $u$  à la fin du chemin
    else
      Soit  $i$  tel que  $u = v_i$ 
      Faire une rotation entre  $v_k$  et  $v_i$ . Le dernier sommet du chemin devient  $v_{i+1}$ .
    end if
  end if
end while

```

On veut montrer que sur un graphe $G_{n,p}$, si $p \geq \frac{40 \ln(n)}{n}$ alors $P(\text{trouver un chemin hamiltonien}) \geq 1 - O(\frac{1}{n})$.

On va commencer par étudier une variante des graphes aléatoires, et on verra plus tard comment on revient à $G_{n,p}$. Dans un premier temps, on fixe une probabilité q . Pour construire $\text{used}(v)$, on considère les $n - 1$ arêtes possibles, et on les insère dans $\text{used}(v)$ avec probabilité $1 - q$. L'ensemble d'arêtes complémentaire est noté $\text{unused}(v)$. Ceci implique en particulier qu'une arête (u, v) peut être placée dans $\text{unused}(u)$ mais pas dans $\text{unused}(v)$, bien que le graphe soit non orienté.

Question 11. Montrer que tant que l'algorithme modifié n'a pas échoué, à chaque étape, tous les sommets ont la même probabilité de devenir le dernier sommet du chemin partiel, peu importe les choix précédents.

On suppose que $q \geq \frac{20 \ln(n)}{n}$.

Soit E_1 l'évènement qui est réalisé lorsque l'algorithme a effectué $3n \ln(n)$ tours de boucle sans échouer et sans trouver de chemin hamiltonien.

Soit F_u l'évènement qui est réalisé lorsque le sommet u n'a pas été visité lors des $2n \ln(n)$ premières étapes de l'algorithme.

Question 12. Montrer que $P(F_u) \leq (1 - \frac{1}{n})^{2n \ln(n)} \leq \frac{1}{n^2}$. En déduire que la probabilité de ne pas avoir visité tous les sommets en $2n \ln(n)$ étapes est majorée par $1/n$.

Question 13. En déduire que $P(E_1) = O(\frac{1}{n})$.

On définit les évènements :

- E_2 : l'algorithme a échoué en moins de $3n \ln(n)$ étapes,
- E_{2a} : il existe un k tel que $\text{unused}(v_k)$ a perdu au moins $9 \ln(n)$ arêtes au cours des $3n \ln(n)$ premières étapes,
- E_{2b} : il y avait au départ un v_k avec $\text{card}(\text{unused}(v_k)) \leq 10 \ln(n)$.

Question 14. Remarquer que $P(E_2) \leq P(E_{2a}) + P(E_{2b})$.

On va majorer chaque terme par $1/n$.

Question 15. Soit X_k la variable aléatoire qui donne le nombre de fois où le sommet v_k s'est retrouvé au bout du chemin partiel. Reconnaitre la loi de X_k . Montrer que $P(X_k \geq 9 \ln(n)) \leq \frac{1}{n^2}$. En déduire que $P(E_{2a}) \leq \frac{1}{n}$.

(Indication. Chernoff en $\ln(n)/3$.)

Question 16. Soit Y_k le nombre d'arêtes d'une liste $\text{unused}(v_k)$ initialement. Reconnaitre la loi de Y_k . En utilisant un raisonnement similaire qu'à la question précédente, montrer que $P(E_{2b}) \leq \frac{1}{n}$.

(Indication. $t = -\ln(n)$.)

On revient désormais à un graphe $G_{n,p}$. On choisit q tel que $p = 2q - q^2$. À partir d'un graphe $G_{n,p}$, on initialise les listes used et unused de la façon suivante :

- l'arête (u, v) est placée dans $\text{unused}(u)$ uniquement avec probabilité $p \frac{q(1-q)}{2q-q^2}$,
- l'arête (u, v) est placée dans $\text{unused}(v)$ uniquement avec probabilité $p \frac{q(1-q)}{2q-q^2}$,
- l'arête (u, v) est placée dans les deux avec probabilité $p \frac{q^2}{2q-q^2}$.

Question 17. Vérifier qu'une arête est placée dans $\text{unused}(u)$ avec probabilité q . Vérifier que c'est indépendant du fait qu'elle soit placée dans $\text{unused}(v)$.

Et donc le résultat précédent s'applique.

Question 18. Conclure.